

基于微内核的自演进 RAN 原型 — 部署与打包文档

版本：Alpha 1.0

英文名称：Microkernel-based Self-Evolving RAN Prototype

适用对象：需要在另一台计算机上部署本系统的运维人员 / 科研人员

最后更新：2026-05-22

目录

- [1. 系统概述](#)
- [2. 环境要求](#)
- [3. 项目目录结构](#)
- [4. 首次部署步骤（详细）](#)
- [5. 启动系统](#)
- [6. 验证部署](#)
- [7. 页面导航与功能说明](#)
- [8. 常见问题与排查](#)
- [9. 打包与分发指南](#)
- [10. 附录：完整依赖清单](#)

1. 系统概述

基于微内核的自演进 RAN 原型 (Microkernel-based Self-Evolving RAN Prototype) 是一个基于 Web 的科研展示与验证平台，核心思想是：保留一个最小、可信、稳定的通信微内核作为系统底座，在此基础上由 AI 按需动态生成、组合、验证并部署通信功能模块。

系统通过 Streamlit 提供交互式仪表盘，共包含 **8 个功能页面**，覆盖从需求输入到自演进优化的完整闭环：

页面	功能
🏠 系统首页	系统定位、核心关键词、能力总览，引导用户进入各功能模块
模块1: 链路自适应	信道变化模拟、KPI 实时曲线、拓扑可视化、数据导出与频段切换
模块2: AI意图解析与方案生成	AI 多智能体编排、自然语言需求理解、候选通信方案生成
模块3: 数字孪生方案验证	接口/约束/链路/故障/评分五步验证流程
模块4: 微内核Runtime运行	模块热加载、50 步执行、KPI 监控、故障注入与安全回退
模块5: SDR半实物验证	IQ 波形生成、AWGN 信道模拟、BER/EVM 测量、频谱分析
模块6: 自演进优化	经验持久化、策略记忆、四类优化建议、多轮迭代演进
端到端全流程演示	AI→孪生→Runtime→SDR→演进 一体化串联演示

技术栈: Python 3.10+ / Streamlit / Plotly / Pandas / NumPy / Pydantic

2. 环境要求

2.1 操作系统

系统	支持情况
Windows 10 / 11 (64-bit)	☑ 完全支持
macOS 12+ (Intel / Apple Silicon)	☑ 完全支持
Linux (Ubuntu 20.04+ / CentOS 7+)	☑ 完全支持

2.2 Python 版本

版本	支持情况
Python 3.9	⚠️ 部分兼容（推荐升级）
Python 3.10	✅ 支持
Python 3.11	✅ 支持
Python 3.12	✅ 推荐
Python 3.13	✅ 推荐（当前开发版本）

注意：不支持 Python 3.8 及以下版本（`pydantic>=2.0` 要求 3.9+）。

2.3 硬件要求

资源	最低要求	推荐配置
CPU	双核 2.0 GHz	四核 3.0 GHz+
内存	4 GB	8 GB+
磁盘	500 MB 空闲	2 GB 空闲（含输出数据）
网络	无需外网（离线运行）	局域网访问需开放端口

2.4 网络端口

端口	用途	是否必须
8501 (TCP)	Streamlit Web 服务	✅ 必须（本地访问）
8501 (TCP)	局域网 / 外网访问	可选（如仅本地使用则无需开放）

3. 项目目录结构

AI RAN Demo/

- └─ LOGO.jpg
- └─ requirements.txt
- └─ install.bat
- └─ start.bat
- └─ package_full.ps1
- └─ 首页设计.md
- └─ 设计思路.txt
- └─ 初步设计.txt
- └─ 详细设计.txt
- └─ 系统主页截图.png
- └─ configs/
 - └─ research_scenarios/
 - └─ p2p_link_adaptation.yaml
 - └─ blockage_recovery.yaml
 - └─ multi_node_cell.yaml
 - └─ jamming_self_evolution.yaml
- └─ uran/
 - └─ __init__.py
 - └─ ai/
 - └─ orchestrator.py
 - └─ agent_base.py
 - └─ intent_agent.py
 - └─ environment_agent.py
 - └─ module_selection_agent.py
 - └─ parameter_gen_agent.py
 - └─ critic_agent.py
 - └─ intent_schema.py
 - └─ plan_schema.py
 - └─ llm_adapter.py
 - └─ twin/
 - └─ sandbox.py
 - └─ validators.py
 - └─ link_simulator.py
 - └─ fault_injection.py
 - └─ scoring.py
 - └─ report.py
 - └─ runtime/
 - └─ microkernel.py
 - └─ module_loader.py
 - └─ monitor.py
 - └─ safety_guard.py
 - └─ state_store.py
 - └─ events.py
 - └─ modules/
 - └─ registry.py
 - └─ base.py
 - └─ modulation.py
 - └─ coding.py
 - └─ harq.py

- # 系统图标
- # Python 依赖清单
- # Windows 一键安装脚本
- # Windows 一键启动脚本
- # PowerShell 完整打包脚本
- # 系统首页 UI 设计文档
- # 原始设计理念
- # 架构/概要/模块设计
- # 实现级设计说明
- # 系统主页截图
- # 科研场景 YAML 配置文件
- # 点对点链路自适应
- # 遮挡恢复场景
- # 多节点小区场景
- # 干扰自演进场景
- # 核心代码包
- # 包初始化
- # 模块2: AI 编排智能体 (11 文件)
- # 主编排器
- # 智能体基类
- # 意图解析智能体
- # 环境分析智能体
- # 模块选择智能体
- # 参数生成智能体
- # 方案审查智能体
- # 意图数据结构
- # 方案数据结构
- # LLM 适配器 (预留)
- # 模块3: 数字孪生沙箱 (6 文件)
- # 沙箱主入口
- # 接口与约束验证
- # 链路仿真器
- # 故障注入
- # 综合评分
- # 验证报告数据结构
- # 模块4: 微内核 Runtime (6 文件)
- # 微内核核心
- # 模块加载器
- # 运行监控
- # 安全守护
- # 状态存储
- # 事件定义
- # 通信模块库 (9 文件)
- # 模块注册中心
- # 模块基类
- # 调制模块
- # 编码模块
- # HARQ 模块

└─ pilot.py	# 导频模块
└─ scheduler.py	# 调度模块
└─ power_control.py	# 功率控制模块
└─ fallback.py	# 回退模块
└─ sdr/	# 模块5: SDR 半实物 (5 文件)
└─ └─ mock_sdr.py	# Mock SDR 实现
└─ └─ adapter_base.py	# SDR 适配器基类
└─ └─ gnuradio_adapter.py	# GNU Radio 适配器
└─ └─ waveform.py	# 波形生成
└─ └─ spectrum.py	# 频谱分析
└─ evolution/	# 模块6: 自演进 (6 文件)
└─ └─ evolution_loop.py	# 演进闭环核心
└─ └─ data_lake.py	# Data Lake 存储
└─ └─ optimizer.py	# 策略优化器
└─ └─ recommender.py	# 优化建议生成
└─ └─ strategy_memory.py	# 策略记忆
└─ └─ experience.py	# 经验记录
└─ dashboard/	# Web 仪表盘 (6 文件)
└─ └─ research_app.py	# ★ 主入口文件 (8 页面路由)
└─ └─ research_controller.py	# 仿真控制器 (含频段切换)
└─ └─ topology_view.py	# 拓扑可视化
└─ └─ kpi_charts.py	# KPI 图表
└─ └─ timeline_view.py	# 事件时间线
└─ └─ report_exporter.py	# 报告导出
└─ sim/	# 仿真核心 (9 文件)
└─ └─ experiment.py	# 实验运行器
└─ └─ adaptation.py	# 链路自适应算法
└─ └─ channel_timeline.py	# 时变信道模型
└─ └─ kpi_model.py	# KPI 估算模型
└─ └─ kpi_recorder.py	# KPI 记录器
└─ └─ link.py	# 链路模型
└─ └─ node.py	# 节点模型
└─ └─ topology.py	# 拓扑模型
└─ └─ scenario_loader.py	# 场景加载器
└─ common/	# 公共工具 (6 文件)
└─ └─ band_config.py	# ★ 频段配置 (n41/n79/自定义)
└─ └─ demo_defaults.py	# 默认参数常量
└─ └─ exceptions.py	# 自定义异常
└─ └─ ids.py	# 唯一 ID 生成
└─ └─ serialization.py	# 序列化工具
└─ └─ time_utils.py	# 时间工具
└─ demo/	# 端到端演示
└─ └─ end_to_end.py	
└─ utils/	
└─ └─ file_utils.py	
└─ tests/	# 测试用例 (11 个文件, 119 个用例)
└─ └─ test_adaptation.py	# 链路自适应决策 (4 用例)
└─ └─ test_ai_orchestrator.py	# AI 编排器 (7 用例)
└─ └─ test_band_config.py	# 频段配置 (15 用例) ★
└─ └─ test_channel_timeline.py	# 信道时间线 (5 用例)
└─ └─ test_dashboard_exporter.py	# 报告导出 (3 用例)

test_digital_twin.py	# 数字孪生（10 用例）
test_end_to_end.py	# 端到端集成（8 用例）
test_evolution.py	# 自演进（12 用例）
test_microkernel.py	# 微内核（30 用例）
test_research_simulation.py	# 科研仿真（5 用例）
test_sdr.py	# SDR 信号处理（15 用例）
outputs/	# 输出目录（运行时自动创建）
figures/	# 图表导出
experiment_logs/	# 实验日志
evolution/	# 演进经验数据
docs/	# 设计文档
系统介绍与用户手册-V2.md	# 系统完整介绍 + 用户手册
系统介绍与用户手册-V2.pdf	# 上述文档 PDF 版
AI_RAN_Demo_完整测试报告-V2.md	# 119 个用例完整测试报告
AI_RAN_Demo_完整测试报告-V2.pdf	# 上述文档 PDF 版
AI_RAN_Demo_部署与打包文档.md	# 本文档
概要设计文档.md	# 系统架构设计
模块设计文档.md	# 模块接口设计
详细设计文档.md	# 类设计文档
测试报告.md	# 模块1 17 个测试用例
备份/	# 历史版本文档

4. 首次部署步骤（详细）

步骤 1：获取项目代码

将整个 `AI RAN Demo` 目录复制到目标计算机的任意位置，例如：

```
C:\Projects\AI RAN Demo\           (Windows)
/Users/username/AI RAN Demo/       (macOS)
/home/username/AI RAN Demo/        (Linux)
```

重要：确保目录路径中**不包含中文或空格**，否则可能导致 Python 导入异常。

步骤 2：安装 Python

Windows 用户：

1. 访问 <https://www.python.org/downloads/> 下载 Python 3.10 ~ 3.13（推荐 3.12）
2. 安装时**勾选** "Add Python to PATH"
3. 打开 PowerShell 或命令提示符，验证安装：

```
python --version
# 应输出: Python 3.12.x 或类似
```

macOS 用户:

```
# 使用 Homebrew 安装 (推荐)
brew install python@3.12
python3 --version
```

Linux 用户:

```
# Ubuntu / Debian
sudo apt update && sudo apt install python3.12 python3.12-venv python3-pip

# CentOS / RHEL
sudo dnf install python3.12 python3-pip
```

步骤 3: 创建虚拟环境 (推荐)

在项目根目录下创建 Python 虚拟环境，避免与系统其他项目冲突:

```
cd "AI RAN Demo"

# 创建虚拟环境
python -m venv venv

# 激活虚拟环境
# Windows PowerShell:
venv\Scripts\Activate.ps1
# Windows CMD:
venv\Scripts\activate.bat
# macOS / Linux:
source venv/bin/activate
```

激活成功后，终端提示符前会出现 `(venv)` 标识。

步骤 4: 安装依赖

```
# 确保在项目根目录且虚拟环境已激活
pip install -r requirements.txt
```

安装过程约 1-3 分钟，会自动安装以下核心依赖：

包名	版本要求	用途
streamlit	>=1.30.0	Web 仪表盘框架
plotly	>=5.18.0	交互式 KPI 图表
pandas	>=2.0.0	数据处理与 CSV 导出
numpy	>=1.24.0	数值计算
pydantic	>=2.0.0	数据结构定义
PyYAML	>=6.0.0	场景配置文件解析
pytest	>=7.0.0	单元测试框架
kaleido	>=0.2.1	Plotly 静态图表导出

步骤 5：验证安装

```
# 运行快速验证
python -c "import streamlit; import plotly; import pandas; import numpy; import pydantic; import yaml; print('全部依赖安装成功!')"
```

如果输出 `全部依赖安装成功!`，则表示环境准备完成。

步骤 6（可选）：使用一键安装脚本

Windows 用户也可以直接双击项目根目录中的 `install.bat`，自动完成虚拟环境创建和依赖安装。

5. 启动系统

5.1 正常启动（推荐）

在项目根目录下执行：

```
streamlit run uran/dashboard/research_app.py --server.port 8501
```

启动成功后，终端会显示：

You can now view your Streamlit app in your browser.

Local URL: `http://localhost:8501`

Network URL: `http://192.168.x.x:8501`

在浏览器中打开 `http://localhost:8501` 即可访问。

启动后默认显示 **系统首页**，左侧边栏包含：

- 系统 LOGO 与品牌标识
- 8 个页面导航
- 频段配置选择器（n41 2.6GHz 100MHz / n79 4.9GHz 100MHz / n41 50MHz / 仿真默认 500kHz / 自定义）

5.2 指定端口启动

```
streamlit run uran/dashboard/research_app.py --server.port 8888
```

5.3 允许局域网其他设备访问

```
streamlit run uran/dashboard/research_app.py --server.port 8501 --server.address 0.0.0.0
```

5.4 无头模式启动（无浏览器弹窗，适合服务器）

```
streamlit run uran/dashboard/research_app.py --server.port 8501 --server.headless true
```

5.5 Windows 快捷启动脚本

双击项目根目录中的 `start.bat` 即可一键完成“激活虚拟环境 + 启动 Streamlit”。

5.6 macOS / Linux 快捷启动脚本

创建 `start.sh`：

```
#!/bin/bash
source venv/bin/activate
streamlit run uran/dashboard/research_app.py --server.port 8501
```

```
chmod +x start.sh
./start.sh
```

6. 验证部署

6.1 浏览器验证

打开 `http://localhost:8501` , 应看到:

- **左侧边栏**: 系统 LOGO、品牌名称"基于微内核的自演进 RAN 原型"、8 个页面导航、频段配置选择器
- **默认显示**: 系统首页 (含系统定位、核心关键词、能力说明)
- **切换到模块1**: 场景选择下拉框 → 页面顶部控制栏 (初始化/重置/单步/全部/自动) → 拓扑图 → KPI 曲线

6.2 运行测试套件

```
cd "AI RAN Demo"
python -m pytest tests/ -v --tb=short
```

预期所有 11 个测试文件 (119 个测试用例) 全部通过, 耗时约 1-2 秒。

6.3 快速功能验证清单

验证项	操作	预期结果
系统首页	打开浏览器访问	显示系统定位与核心能力介绍
模块1 链路自适应	选择场景 → 点击页面顶部「初始化」→ 「全部」	显示 SNR/BLER/吞吐量曲线与事件时间线
模块1 频段切换	左侧边栏切换频段 → 重新运行仿真	KPI 数值随频段参数变化
模块1 数据导出	点击导出 CSV/JSON/Markdown	outputs/ 目录生成文件
模块2 AI编排	输入需求 → 点击「生成方案」	显示环境信息 + 候选方案列表
模块3 数字孪生	输入需求 → 点击验证	显示五步验证报告 + ACCEPT/REJECT 决策
模块4 Runtime	加载方案 → 运行	显示 KPI 汇总 + 活跃模块列表
模块4 故障回退	点击注入故障	自动切回 BPSK 保底模式
模块5 SDR验证	切换调制方式 → 运行	显示 BER / EVM / 频谱图
模块6 自演进	点击「启动演进闭环」	显示优化建议 + 历史经验统计
端到端演示	点击「运行全流程」	完整六阶段结果展示

7. 页面导航与功能说明

7.1 系统首页

启动后默认显示。展示系统的定位、核心关键词（微内核 / AI 受约束生成 / 全链路验证 / 安全回退 / 持续演进）和能力总览，引导用户通过左侧导航进入各功能模块。

7.2 模块1: 链路自适应

验证通信系统最基本的自适应能力。

- **频段配置：**左侧边栏底部提供 5 种频段方案，切换后所有 KPI 计算自动使用新参数

- **场景选择**: 下拉选择 `p2p_link_adaptation.yaml` 等 4 个预设场景
- **页面顶部控制栏**: 全部控制按钮集成在页面顶部一行
 - 「初始化」— 加载场景配置, 初始化仿真环境
 - 「重置」— 回到初始状态
 - 「单步」— 每次执行一步, 观察逐步变化
 - 「全部」— 一次性运行全部仿真步骤
 - 「自动」— 自动循环播放 (适合展厅无人值守)
 - 间隔滑块 — 调节自动播放步进速度
- **可视化**: 网络拓扑图、SNR/SINR 曲线、BLER 曲线、吞吐量曲线、时延曲线、自适应参数变化、事件时间线
- **导出**: CSV / JSON / HTML / Markdown 报告

7.3 模块2: AI意图解析与方案生成

AI 多智能体协作生成通信方案。

- 用户输入自然语言需求 (如"城区 SNR=10dB, 要求 BLER<5%, 吞吐量>500kbps")
- 系统自动解析意图 (场景类型、优先级、性能目标)
- 分析环境 (SNR、移动速度、干扰水平)
- 生成 2-3 套候选通信方案 (调制方式、编码方案、HARQ 配置等)
- 显示 5 个 Agent 的编排执行轨迹

7.4 模块3: 数字孪生方案验证

在沙箱中验证候选方案。

- 接口兼容性检查 (方案是否包含所有必需字段)
- 约束条件验证 (参数是否在合法范围内)
- 链路级仿真 (BLER、吞吐量、时延、频谱效率、能耗)
- 故障注入测试 (SNR 骤降、突发干扰下的鲁棒性)
- 综合评分 (0-100) 与最终决策 (ACCEPT / NEED_REVISE / REJECT)

7.5 模块4: 微内核Runtime运行

加载方案到微内核并执行。

- 模块动态加载与监控
- 50 步执行 + KPI 实时记录
- 安全守护 (异常检测与回退)

- 故障注入测试 → 自动切换 BPSK 保底模式

7.6 模块5: SDR半实物验证

模拟软件定义无线电收发。

- IQ 波形生成 (BPSK / QPSK / 16QAM / 64QAM)
- AWGN 信道模拟
- BER / EVM / 丢包率测量
- 功率谱密度 (PSD) 可视化

7.7 模块6: 自演进优化

闭环持续优化。

- 基于历史 KPI 数据生成优化建议
- 经验持久化 (JSONL 格式)
- 策略记忆与推荐
- 多轮迭代演进

7.8 端到端全流程演示

模块2 → 模块3 → 模块4 → 模块5 → 模块6 一体化串联，适合科研汇报展示。

用户输入一句自然语言需求 → 系统自动完成 AI 生成方案 → 数字孪生验证 → 微内核执行 → SDR 演示 → 演进优化 → 经验存储 的全流程。

8. 常见问题与排查

Q1: `streamlit: command not found` 或 无法识别 `streamlit`

原因: 虚拟环境未激活，或 streamlit 未安装。

解决:

```
# 激活虚拟环境后再安装
venv\Scripts\activate          # Windows
source venv/bin/activate       # macOS/Linux
pip install -r requirements.txt
```

也可使用 `python -m streamlit run ...` 替代 `streamlit run ...`。

Q2: `ImportError: cannot import name 'xxx'`

原因：项目未在正确的目录下运行，或 `__pycache__` 缓存冲突。

解决：

```
# 确保在项目根目录（包含 uran/ 的目录）
cd "AI RAN Demo"

# 清除 Python 缓存
# Windows PowerShell:
Get-ChildItem -Recurse -Directory -Filter "__pycache__" | Remove-Item -Recurse -Force

# macOS / Linux:
find . -type d -name "__pycache__" -exec rm -rf {} +

# 重新运行
streamlit run uran/dashboard/research_app.py
```

Q3: 端口 8501 被占用

错误信息： `Address already in use` 或 端口被占用

解决：使用其他端口启动

```
streamlit run uran/dashboard/research_app.py --server.port 8502
```

Q4: 页面打开后空白 / 长时间加载

原因：首次访问时 Streamlit 需要编译页面，可能需 10-20 秒。

解决：等待 30 秒后刷新浏览器（F5）。页面加载后无数据时，需先点击对应模块的操作按钮。

Q5: 图表不显示

原因：Plotly 渲染需要浏览器支持 JavaScript。

解决：

- 确保浏览器未禁用 JavaScript
- 推荐使用 Chrome / Edge / Firefox 最新版

- 不支持 IE 浏览器

Q6: 虚拟机中运行时性能差

原因：虚拟机 CPU / 内存分配不足。

解决：

- 分配至少 4 GB 内存、2 个 CPU 核心
- 关闭虚拟机中不必要的后台程序

Q7: `kaleido` 安装失败

原因：`kaleido` 用于导出静态图表，某些系统可能安装失败。

解决：

```
# 方式1: 使用 conda
conda install -c conda-forge python-kaleido

# 方式2: 跳过 kaleido (不影响核心功能, 仅影响图表导出为 PNG)
# 编辑 requirements.txt, 删除 kaleido 一行后重新安装
```

Q8: 频段切换后 KPI 未变化

原因：需要重新运行仿真或重新加载方案才能应用新频段参数。

解决：在左侧边栏切换频段后，重新点击对应模块的操作按钮（如模块1 的「初始化」→「全部」）。

9. 打包与分发指南

9.1 打包方式对比

方式	优点	缺点	适合场景
ZIP 压缩	简单、通用	不含 Python 环境	目标电脑已有 Python
自解压 EXE	Windows 双击即可	仅限 Windows	Windows 分发给非技术人员
Git 仓库	版本管理、增量更新	需要 Git 客户端	开发团队内部分发
PyInstaller 打包	单文件 EXE、无需 Python	体积大、启动慢	完全无 Python 环境的电脑

9.2 方式一：ZIP 压缩打包（推荐）

打包步骤（源电脑）：

Windows PowerShell：

```
cd "E:\AI_RAN_Demo"

# 1. 清理 Python 缓存和临时文件
Get-ChildItem -Recurse -Directory -Filter "__pycache__" | Remove-Item -Recurse -Force
Get-ChildItem -Recurse -Directory -Filter ".pytest_cache" | Remove-Item -Recurse -Force
Remove-Item -Recurse -Force outputs -ErrorAction SilentlyContinue

# 2. 打包（排除虚拟环境和缓存，目标电脑会自己创建）
Compress-Archive -Path @(
    "configs",
    "docs",
    "tests",
    "uran",
    "requirements.txt",
    "start.bat",
    "install.bat",
    "LOGO.jpg"
) -DestinationPath "AI_RAN_Demo_$(Get-Date -Format 'yyyyMMdd').zip" -Force

Write-Host "打包完成：AI_RAN_Demo_$(Get-Date -Format 'yyyyMMdd').zip"
```

macOS / Linux：


```
cd "/path/to/AI RAN Demo"

# 清理缓存
find . -type d -name "__pycache__" -exec rm -rf {} +
find . -type d -name ".pytest_cache" -exec rm -rf {} +
rm -rf outputs

# 打包
zip -r "AI_RAN_Demo_$(date +%Y%m%d).zip" \
    configs/ docs/ tests/ uran/ requirements.txt start.bat install.bat LOGO.jpg

echo "打包完成: AI_RAN_Demo_$(date +%Y%m%d).zip"
```

解包部署步骤（目标电脑）：

1. 将 `.zip` 文件传输到目标电脑（U盘、局域网共享、云盘等）
2. 解压到目标目录（路径不要含中文和空格）
3. 双击 `install.bat`（Windows）或按照本文档 [第4节](#) 手动部署

9.3 方式二：一键完整打包脚本

在源电脑的 PowerShell 中运行：

```
cd "E:\AI RAN Demo"
.\package_full.ps1
```

脚本会自动执行以下步骤：

1. 清理 `__pycache__` 和 `outputs` 缓存
2. 创建带时间戳的分发目录
3. 复制 `uran/`、`configs/`、`tests/`、`docs/`、`requirements.txt`
4. 生成 `install.bat`（一键安装）和 `start.bat`（一键启动）
5. 生成 `README.md` 快速入门说明
6. 打包为 `AI_RAN_Demo_yyyyMMdd_HHmmss.zip`

输出文件位置：`E:\AI RAN Demo\dist\AI_RAN_Demo_yyyyMMdd_HHmmss.zip`

9.4 方式三：使用 PyInstaller 打包为独立 EXE

注意：PyInstaller 打包体积较大（约 300-500 MB），启动较慢，仅在目标电脑完全无法安装 Python 时使用。

```
# 1. 安装 PyInstaller
pip install pyinstaller

# 2. 创建入口脚本 run_app.py
```

创建 `E:\AI_RAN_Demo\run_app.py` :

```
import sys
import os

os.chdir(os.path.dirname(os.path.abspath(__file__)))
sys.path.insert(0, os.getcwd())

import streamlit.web.cli as stcli

if __name__ == "__main__":
    sys.argv = [
        "streamlit", "run",
        "uran/dashboard/research_app.py",
        "--server.port=8501",
        "--server.headless=true",
        "--browser.serverAddress=localhost",
    ]
    sys.exit(stcli.main())
```

```
# 3. 打包
pyinstaller --onefile --name AI_RAN_Demo --add-data "uran;uran" --add-data
"configs;configs" run_app.py
```

生成的 `dist/AI_RAN_Demo.exe` 可直接运行。

9.5 传输方式建议

方式	适用大小	速度
USB 闪存盘	任意	快
局域网共享文件夹	任意	快
云盘（百度云 / OneDrive / 阿里云盘）	< 4 GB	取决于带宽
微信 / QQ 文件传输	< 2 GB	取决于带宽
邮件附件	< 50 MB	不推荐本项目

附录：完整依赖清单

A.1 直接依赖（requirements.txt）

```
streamlit>=1.30.0
plotly>=5.18.0
pandas>=2.0.0
numpy>=1.24.0
pydantic>=2.0.0
PyYAML>=6.0.0
pytest>=7.0.0
kaleido>=0.2.1
```

A.2 间接依赖（自动安装，无需手动处理）

包名	来源	用途
altair	streamlit 依赖	声明式图表
Jinja2	streamlit 依赖	HTML 模板渲染
protobuf	streamlit 依赖	数据序列化
pydeck	streamlit 依赖	地图可视化
pillow	streamlit 依赖	图像处理
tenacity	plotly 依赖	重试机制
packaging	多库依赖	版本号比较
tzdata	pandas 依赖	时区数据
annotated-types	pydantic 依赖	类型注解增强

A.3 版本锁定建议

如需生产环境部署，建议将 requirements.txt 替换为精确版本：

```
pip freeze > requirements-lock.txt
```

目标电脑使用 `pip install -r requirements-lock.txt` 安装，确保依赖版本与源环境一致。

文档版本: V2.0

适用版本: Alpha 1.0

上一版本: V1.0 (AI-Native RAN 科研演示系统)

更新内容: 系统名称更新、新增系统首页与频段配置说明、页面从 7 扩展到 8、测试用例更新至 119、目录结构完整更新、一键安装/启动脚本更新

项目名称: 基于微内核的自演进 RAN 原型 (Microkernel-based Self-Evolving RAN Prototype)